DIGITAL FUNDAMENTALS & COMPUTER ARCHITECTURE

Memory System

Module 5

The Memory System – Basic Concepts

A computer memory system refers to the entire hierarchy of data-storage components used to store instructions and data.

The memory system plays a crucial role in system performance and determines how fast the processor can access information.

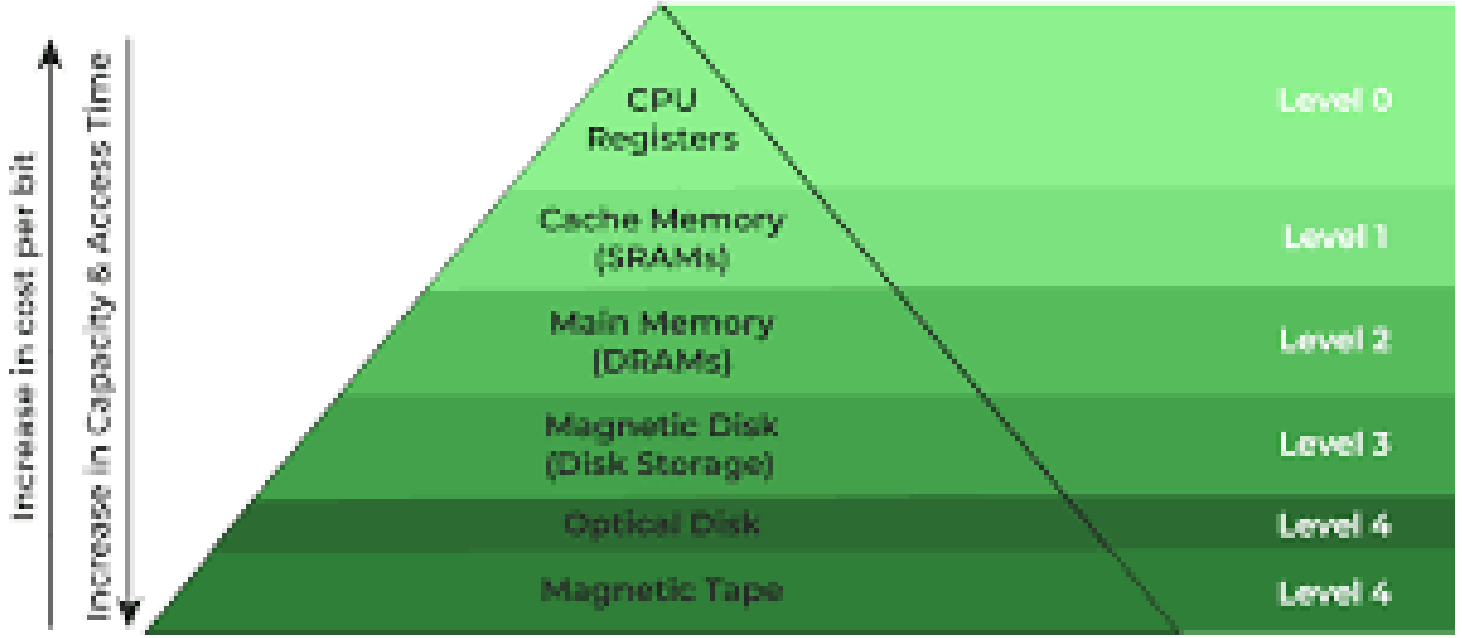
Key Concepts

- **Memory Hierarchy**
  To balance speed, cost, and size, memory in a computer is arranged hierarchically:
  - **Registers** (fastest, smallest, most expensive)
  - **Cache Memory**
  - **Main Memory (RAM)**
  - **Secondary Storage (Hard disk, SSD)**
  - **Tertiary Storage (Cloud, external drives)**

  As we move down the hierarchy, **speed decreases**, **size increases**, and **cost per bit decreases**.

How it works

- **CPU request:** When the CPU needs a piece of data, it first checks the fastest level of memory, the registers.
- **Cache check:** If the data is not in registers, the CPU moves to the cache (L1, L2, etc.). Caches are small but very fast memory, often holding frequently used data.
- **Cache hit/miss:**
  - **Hit:** If the data is found in the cache, it is a "cache hit," and the data is retrieved quickly.
  - **Miss:** If the data is not in the cache, it's a "cache miss," and the system moves to the next, slower level.
- **Main memory and secondary storage:** The system then checks the main memory **(RAM) and finally secondary storage** (like SSDs or hard drives).
- **Data transfer:** When data is retrieved from a slower level, a block of it is transferred to the faster level above it. This is because, due to spatial locality, the CPU is likely to need the data that is near the requested data soon after.
- **Repeat:** The CPU continues to work with the data now stored in the faster memory level until it is needed elsewhere. This process is repeated for all data requests.



Memory Hierarchy Design

- **Access Methods**
  - **Sequential Access**: Data is accessed in a specific order (e.g., magnetic tape).
  - **Random Access**: Any memory location can be accessed directly (e.g., RAM).
  - **Direct Access**: Combination of sequential and random (e.g., hard disks).
  - **Associative Access**: Access based on content, not address.
- **Memory Characteristics**
  - **Volatile vs Non-volatile: Volatile** memory is temporary and requires a continuous power supply to retain data, while **non-volatile memory** is permanent and keeps its data even after the power is turned off.
  - RAM is volatile; ROM and flash are non-volatile.
  - **Read–Write vs Read-Only**: RAM is R/W; ROM is mostly read-only.
- **Performance Parameters**
  - **Access Time**: Time to read/write a location.
  - **Cycle Time**: Minimum time between two memory operations.
  - **Transfer Rate**: Rate at which data is transferred to CPU.
  - **Bandwidth**: Overall data-handling capability.

- The memory system ensures fast, reliable, and efficient data access to meet the CPU's speed demands.
- Main memory is the **primary storage** of a computer system used to store data and instructions that the CPU needs *immediately*. Main memory is also commonly called **RAM (Random Access Memory)** because the processor can access any memory location directly in the same amount of time.

Types of RAM

Static RAM

Dynamic RAM

Static RAM (SRAM)

- Static RAM is a type of semiconductor memory that stores each bit of data using a **bistable Flip-flop circuit** made of transistors. It retains data **as long as power is supplied** and does not require periodic refresh.

- A **bistable flip-flop circuit** is a digital memory circuit that has **two stable states** and is used to store **one bit of information**. It remains stable until triggered to change state.

Bi = two

Stable = stable states
It can stay in:

State 1 → represents binary **1**

State 0 → represents binary **0**

Working

- Once data is written into the flip-flop, it remains stable.
- No refreshing is required, so access is very fast.

Dynamic RAM (DRAM) stores data using **capacitors** that must be refreshed periodically. DRAM can work in two ways based on timing control:

- **Asynchronous DRAM**
- **Synchronous DRAM (SDRAM)**

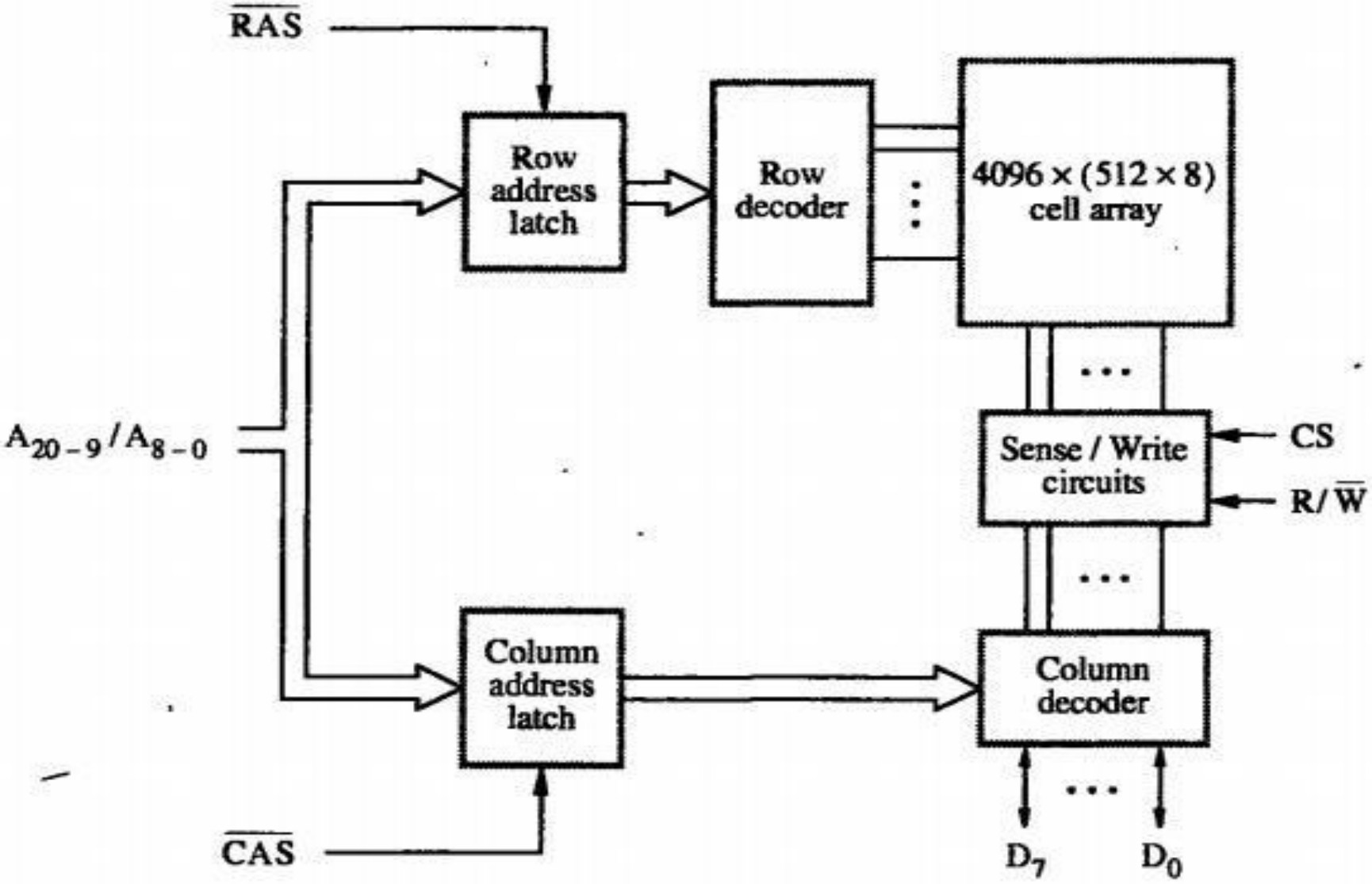
Asynchronous DRAM

- Asynchronous DRAM works **without a clock signal**.
- The CPU must **wait** for DRAM to complete each operation because the memory does not run in sync with the processor clock.

Key Features

- **No clock signal** – operations happen based on signal changes.
- **Slower** because the CPU has to wait for memory responses.
- Timing is **not predictable** since it depends on individual signals.
- Used in **older computers and devices.**

Internal Organization



DRAM Cell Organization

- Each row contains 4-bit cells divided into **512 groups of 8 bits**.
- To access a byte, a **21-bit address** is required:
  - **12 bits** → Select a row
  - **9 bits** → Select a group of 8 bits in that row
- Address division:
  - $\mathbf{A_8–A_0}$ → *Row address*
  - $\mathbf{A_{20}–A_9}$ → *Column address*

Read/Write Operation Sequence

1. Row Addressing

- Row address is applied **first**.
- Loaded into **Row Address Latch**.
- Triggered by **RAS (Row Address Strobe)** signal.
- When a row is selected, **all cells in that row are read and refreshed**.

2. Column Addressing

- After row address latching, column address is applied.
- Loaded into **Column Address Latch** using **CAS (Column Address Strobe)**.
- This selects the required group of 8 bits (sense/write circuits).

Data Transfer

Read Operation

R/W = 1

Output values of the selected cells are placed on the data lines **D0–D7**.

Write Operation

R/W = 0

Data on **D0–D7** is written into the selected memory cells.

Role of RAS and CAS

RAS and **CAS** are *active low* signals.

When they go from **high → low**, the address is latched.

They control:

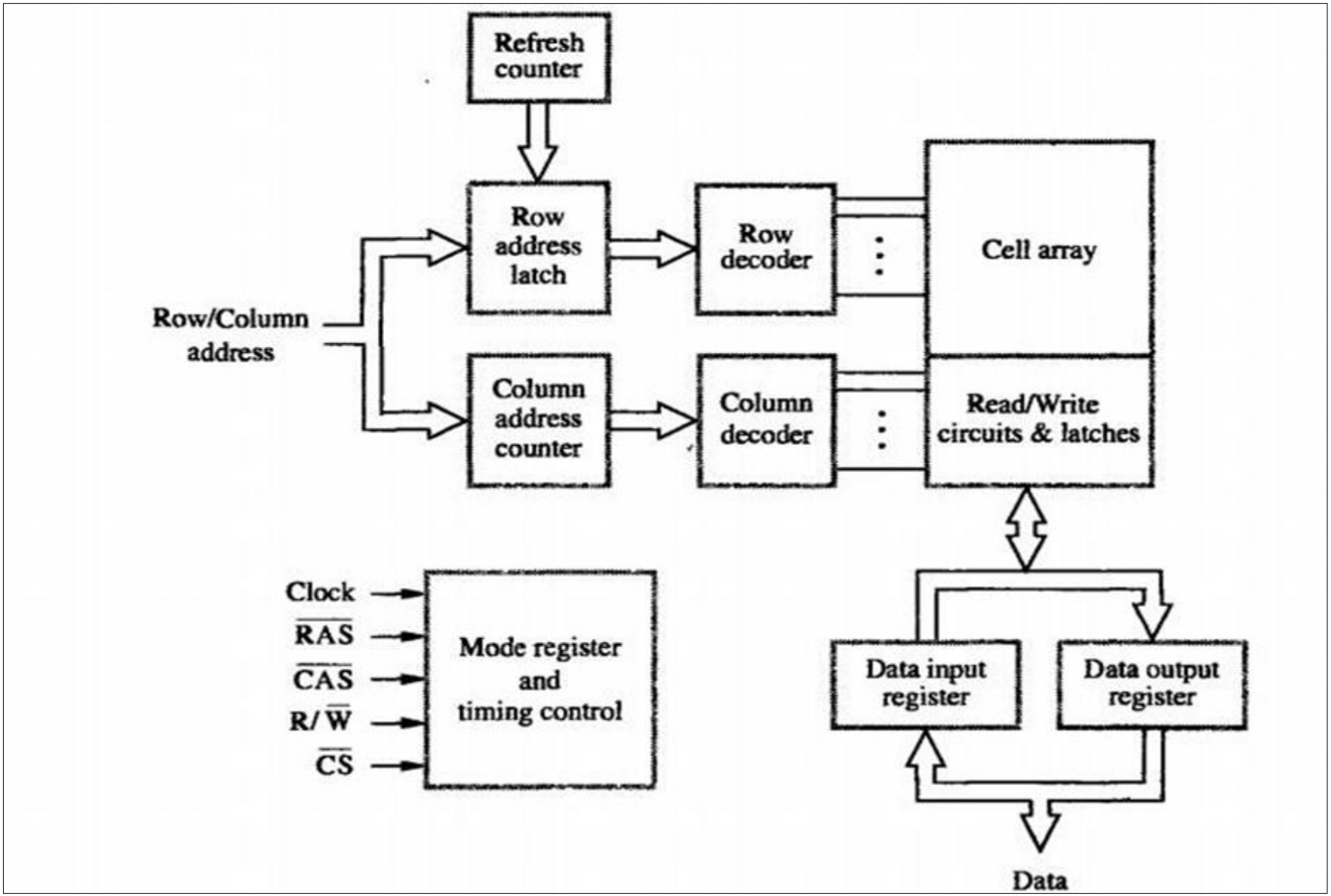
- Row selection
- Column selection
- Timing of data transfer

- DRAM cells lose charge over time → they must be **refreshed periodically**.
- Refresh operation reads and rewrites cells automatically.
- Performed by a **specialized memory controller**.

Timing and Delays

- Processor must consider the **delay** in DRAM operations.
- Because timing depends on RAS and CAS control and not on a clock,
  This type of memory is called **Asynchronous DRAM**.

Synchronous DRAM



Row/Column Address Input

- CPU or memory controller provides the **row and column address**.
- The address is used to locate the **exact memory cell** in the DRAM's 2D array.
- SDRAM splits the address into **row** and **column** parts for faster access.

Row Address Latch & Row Decoder

- **Row Address Latch:** Temporarily stores the row address.
- **Row Decoder:** Activates the selected row in the **cell array**.
- **Effect:** All cells in the selected row are connected to the **read/write circuitry and sense amplifiers**.

Column Address Counter & Column Decoder

- **Column Address Counter:** Stores the column address. Can also **auto-increment** for burst reads.
- **Column Decoder:** Selects the specific columns in the row that need to be read/written.

Cell Array

- Organized as **rows × columns**.
- Each cell stores **1 bit** as a charge in a capacitor.
- Accessed via **row decoder + column decoder**.

Read/Write Circuits & Latches

- Each sense amplifier is connected to a **latch**.
- **Read Operation:** Entire row is loaded into latches.
- **Write Operation:** Data from input register is written into the selected cells.
- Latches act as a **buffer**, speeding up data transfer to/from the CPU.

Data Input and Data Output Registers

- **Data Input Register:** Temporarily stores data from CPU before writing to memory.
- **Data Output Register:** Temporarily holds data read from memory **before sending it to the CPU**.
- Enables **synchronized data transfer** with the system clock.

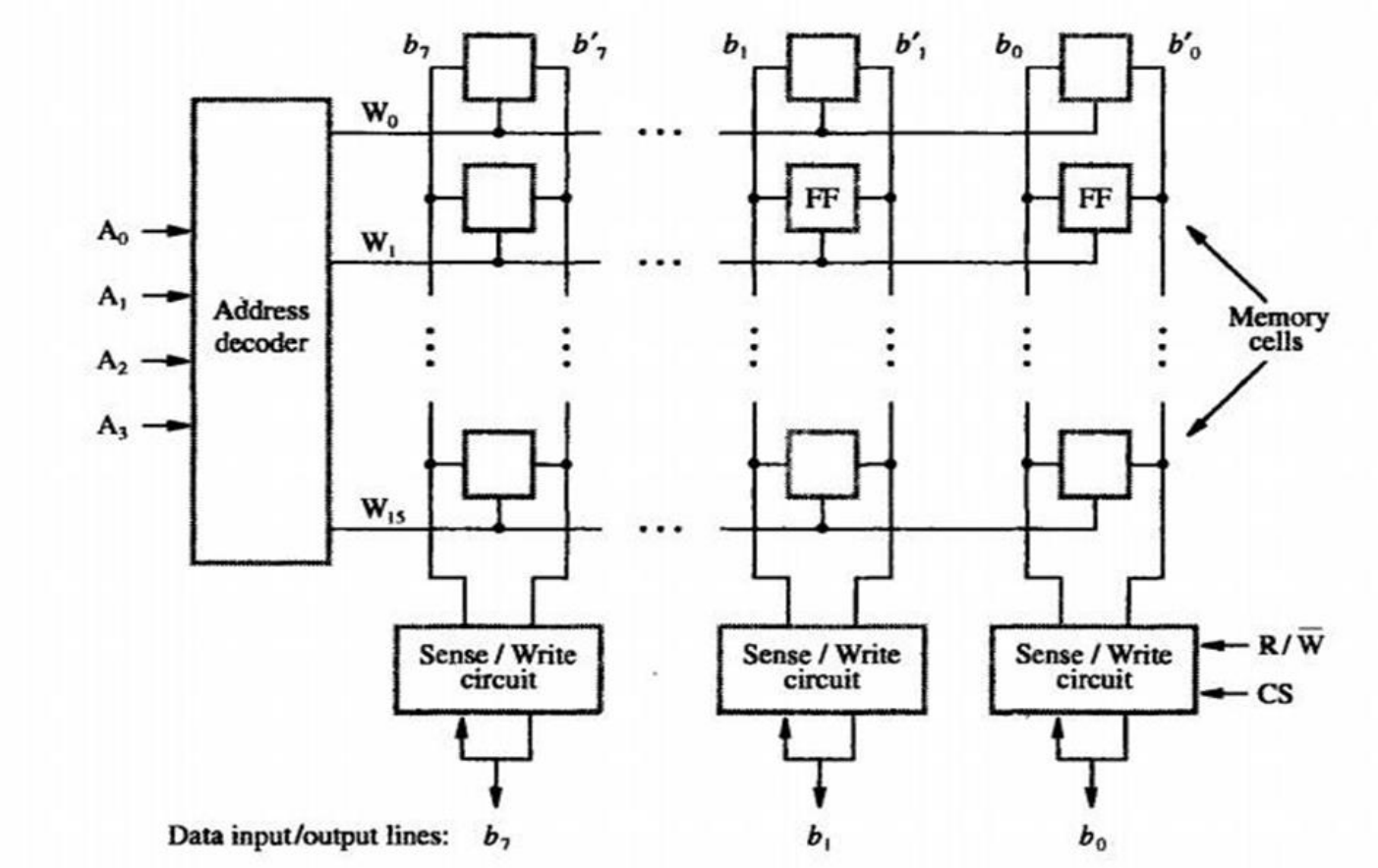
Mode Register & Timing Control

- Controls SDRAM operation (read/write modes, burst length, CAS latency, etc.)
- Inputs include:
  - **Clock:** Synchronizes all operations.
  - **RAS (Row Address Strobe):** Signals row selection.
  - **CAS (Column Address Strobe):** Signals column selection.
  - **R/W:** Read or Write command.
  - **CS:** Chip select.
- Timing control ensures **all operations follow clock cycles**, making SDRAM predictable and faster.

Refresh Counter

- DRAM cells **lose charge over time**.
- Refresh counter periodically refreshes rows to **prevent data loss**.
- Controlled automatically by SDRAM timing logic.

INTERNAL ORGANIZATION OF MEMORY CHIPS:



Address Inputs and Decoder

- **Inputs:** A0,A1,A2,A3 are the **address lines** coming from the CPU.
- **Address Decoder:**
  - The decoder takes the binary address and **activates only one word line** W0,W1,…,W15 corresponding to that address.
  - Example: If the address is 0101, only W5 is activated, selecting the fifth row of memory cells.

Memory Array

- The memory array is organized in **rows and columns**:
  - **Rows:** Each row is connected to a **word line** W0 to W15
  - **Columns:** Each column corresponds to **data lines** b0,b1,…,b7
- **Memory Cells:**
  - Each intersection of a word line and a bit line contains a **memory cell**, which can be:
    - **SRAM:** A flip-flop (FF) storing a single bit
    - **DRAM:** A capacitor and access transistor storing a single bit

Sense/Write Circuits

- Each column has a **sense/write circuit** connected to the **data input/output lines**.
- Functions:
  - **Read:** Detects the small voltage from the selected memory cell and drives it to the data lines.
  - **Write:** Applies voltage to store the input data into the selected cell.

Data Input/Output Lines

- b0,b1,…,b7 are the **data lines**, usually corresponding to **8-bit data bus**.
- The sense/write circuits ensure that **only the selected row's cells** transfer data to these lines.
- **Control Signals**
- **R/W (Read/Write):** Determines the operation:
  - 0 → Read
  - 1 → Write
- **CS (Chip Select):** Enables or disables the memory chip.

Operation Summary

- CPU sends **address** and **control signals** (R/W, CS).
- **Address decoder** selects a row (word line).
- The **sense/write circuits** interact with the memory cells in the selected row.
- Data is **read from or written to** the column lines.
- Output appears on **data bus** b0 to b7.

Key Points

- **Word lines** select rows.
- **Bit lines and sense/write circuits** handle data transfer.
- **Flip-flops or capacitors** are the actual storage elements.
- This structure is **highly regular**, allowing scalable memory sizes.

Cache Memories

- **Early processors (1950–1980)**:
  - CPU speed ≈ Main Memory speed → no major bottleneck.
- **Later processors (1970 onwards)**:
  - CPU became **much faster** than main memory.
  - Result: Processor **waits idle** for data → performance gap.

Solution → Introduce **Cache Memory** between CPU and Main Memory.

Cache = small, fast memory placed between **CPU and RAM**.

- Uses **temporal and spatial locality**:
  - **Temporal locality** → recently used data likely to be used again soon.
  - **Spatial locality** → nearby data likely to be used soon.
- When CPU requests data:
  - **Cache Hit** → Data found in cache → fast access.
  - **Cache Miss** → Data not in cache → fetched from main memory (slow).

Cache Operations

- **On a Cache Miss**:
  - Data fetched from main memory and stored in cache line.
  - Replacement policy used if cache is full (LRU, FIFO, Random).
- **On a Cache Hit**:
  - Data supplied immediately to CPU → very fast.

Mapping Function /Cache Mapping Techniques

- **Mapping function** = Specifies the **correspondence between main memory blocks and cache lines**.
- When the cache is full and a new block must be loaded, the **Replacement Algorithm** decides which block should be removed (e.g., LRU, FIFO, Random).
- Three main mapping techniques define cache organization:
  - **Direct Mapping**
  - **Associative Mapping**
  - **Set-Associative Mapping**

Direct Mapping

- Direct mapping is the **simplest cache mapping technique**, where each block of main memory maps to **exactly one cache line**.

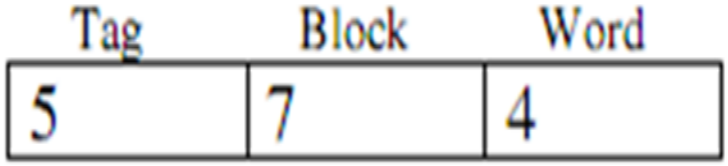
Address Division in Direct Mapping

A memory address is divided into **three fields**:

- **Word Field (Lower Order Bits)**
  - Specifies the exact word within a block.
  - For a block size of 16 words → needs **4 bits** (since $2^4 = 16$).
- **Block Field (Middle Bits)**
  - Identifies the cache line where the block must be placed.
  - Cache has 128 blocks → requires **7 bits** (since $2^7 = 128$).
- **Tag Field (Higher Order Bits)**
  - Stores the remaining higher-order address bits.
  - Used to distinguish between different main memory blocks that map to the same cache line.
  - Here → 16-bit memory address − (7 + 4) = **5 bits for tag**

Working of Direct Mapping

- When the CPU generates a memory address:
  - **Word field** selects the word inside the block.
  - **Block field** directly maps to a particular cache line.
  - **Tag field** is compared with the stored tag to verify if the block is the correct one.
- If the tag matches → **Cache Hit** (data is read from cache).
- If the tag mismatches → **Cache Miss** (data is fetched from main memory and placed in that cache line).



How It Works

1. Main Memory and Cache Setup

- Assume the cache has **N blocks**.
- Main memory is divided into blocks numbered sequentially: 0, 1, 2, 3, …
- Cache is smaller than main memory, so multiple main memory blocks can map to the same cache block.

2. Mapping Formula

- The formula for **direct mapping** is:

$$\text{Cache Block} = i \bmod N$$

Where:

- iii = main memory block number
- NNN = number of cache blocks
- This means each main memory block is assigned **exactly one cache block** based on the remainder when its number is divided by N.

Example

Suppose:

Cache has **128 blocks** (N=128N = 128N=128)

Then mapping works like this:

Observation:

•Blocks 0, 128, 256… all map to cache block 0.

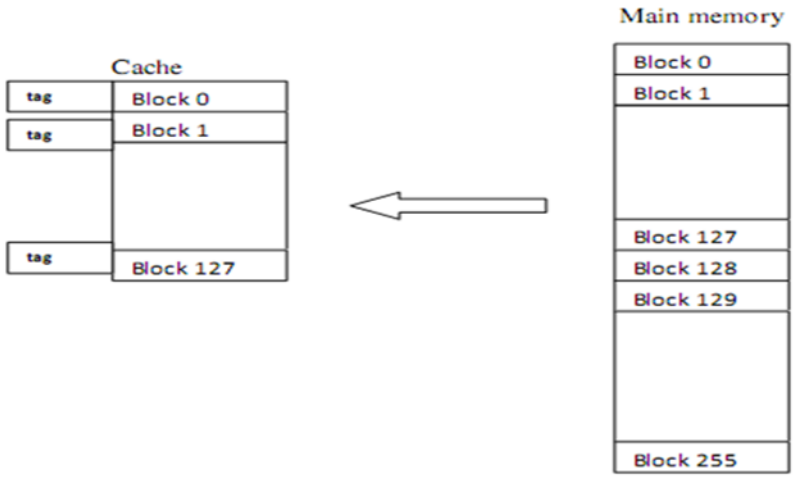
•Blocks 1, 129, 257… all map to cache block 1.

•This pattern repeats for all cache blocks.

Main Memory Block	Calculation	Cache Block
0	0 mod 128	0
128	128 mod 128	0
256	256 mod 128	0
1	1 mod 128	1
129	129 mod 128	1

4. Implications

- **One location per main memory block**
  - Each memory block can only go to one specific cache block.
- **Cache conflicts**
  - If another main memory block maps to the same cache block, it **replaces the existing block**.
  - Example: If block 256 is loaded into cache block 0, it will replace block 0 if it was already there.
  - This is called a **cache conflict**, which can reduce cache efficiency.



Associative Mapping

A main memory block can be placed in **any block** of the cache. There is no fixed location as in direct mapping.

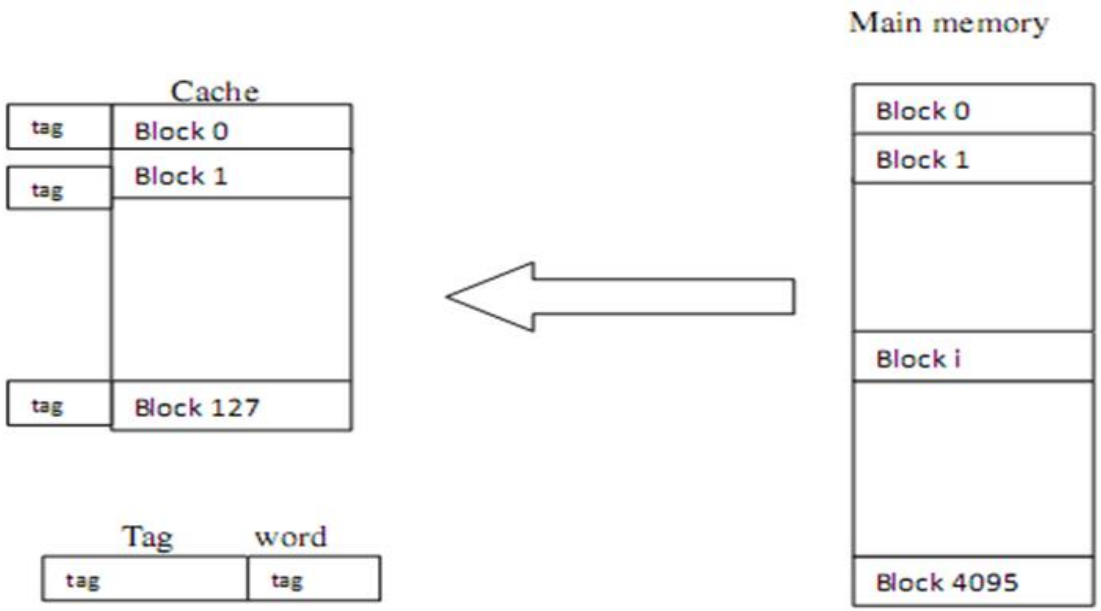
Provides flexibility and reduces cache conflicts.

How It Works

- **Main Memory and Cache:**
  - Cache blocks can store **any main memory block**.
  - This means a memory block doesn't have a predetermined location; it can go anywhere in cache.
- **Tag Bits:**
  - Since any memory block can occupy any cache block, each cache block must store **tag bits** that uniquely identify which memory block it contains.
  - Example: If the memory address is 32 bits and the cache block offset is 20 bits, the **remaining 12 bits** are used as the **tag**.
- **Searching in Cache:**
  - When the CPU requests data, the **tag bits** from the memory address are compared with the tag bits of **all cache blocks simultaneously**.
  - If a match is found, the block is present (**cache hit**); otherwise, it's a **cache miss**.

Example

- Suppose cache has 8 blocks.
- Memory blocks 5, 15, 25, etc., can be stored in **any of the 8 cache blocks**, not just a fixed one.
- Each cache block stores **tag bits** to identify which memory block it contains.



Set-Associative Mapping

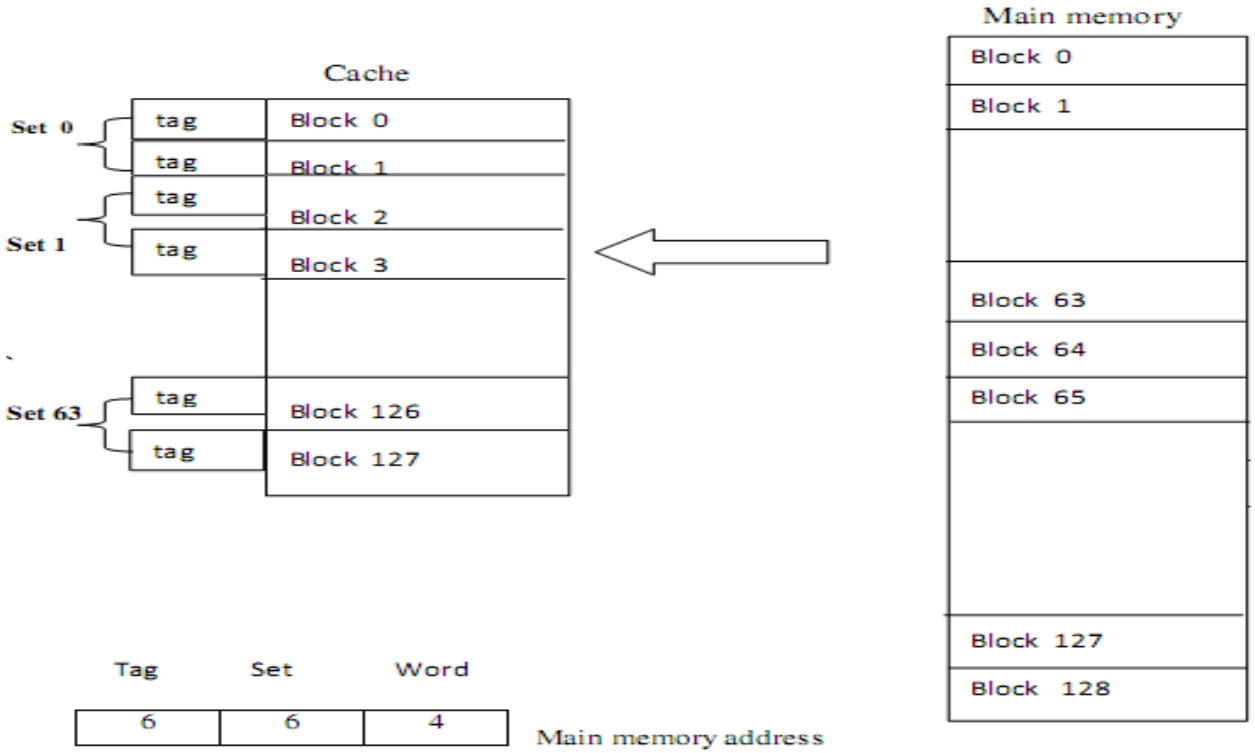
- Hybrid between **Direct Mapping** and **Fully Associative Mapping**
- Cache is divided into **sets**, each containing multiple blocks. A main memory block maps to **one specific set**, but it can occupy **any block within that set**.
- Combines the simplicity of direct mapping with the flexibility of associative mapping.

How It Works

- **Cache Structure:**
  - Suppose cache has N blocks and is **2-way set associative**.
  - The cache is divided into N/2 sets, each containing 2 blocks.
- **Mapping Formula:**
  - Each memory block maps to **one set**:
- Cache Set=Block Number mod Number of Sets
  - Within the set, the memory block can occupy **any of the two blocks**.
- **Tag Bits:**
  - Each block stores **tag bits** to identify the memory block.
  - For a **2-way set-associative cache**, the tag is compared with the tags of the **two blocks in the set**.

Key Features

- Reduces **conflict misses** compared to direct mapping.
- Simpler **associative search** compared to fully associative cache (search only within a set, not entire cache).
- Hardware complexity is **moderate**—not as simple as direct mapping, not as complex as fully associative.



Virtual Memory

Virtual memory is a memory management technique that allows a computer to execute programs even when the program size is larger than the available physical main memory (RAM).

It achieves this by using secondary storage (hard disk) as an extension of RAM and loading only required parts of a program into main memory.

Example

- A program needs **12 KB** of memory to run.
- Physical memory (RAM) available = **8 KB**.
- Page size = **4 KB**.
- Disk (secondary storage) is used to store pages not currently in RAM.

Step 1: Divide program into pages

Program size = 12 KB, Page size = 4 KB → 12 ÷ 4 = 3 pages.

- Page 0 → first 4 KB
- Page 1 → next 4 KB
- Page 2 → last 4 KB

Step 2: Divide RAM into frames

RAM = 8 KB, Frame size = 4 KB → 8 ÷ 4 = 2 frames.

- Frame 0 → can hold 1 page
- Frame 1 → can hold 1 page

Step 3: Load pages into RAM as needed

Page #	Frame #	Status
0	0	In RAM
1	1	In RAM
2	-	On Disk (not in RAM)

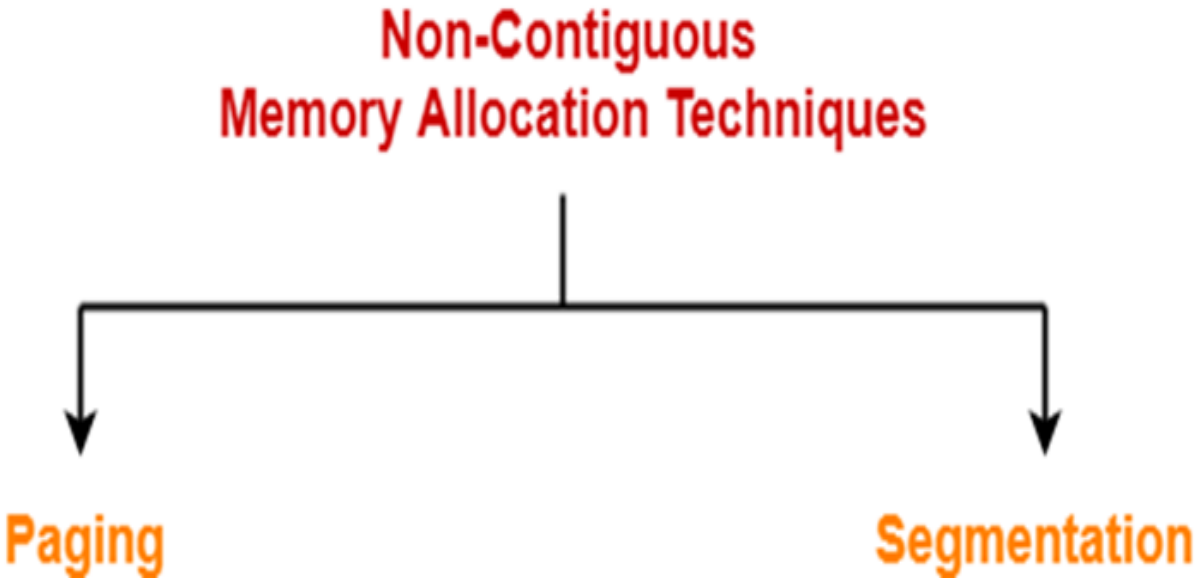
- Program starts execution. Pages 0 and 1 are in RAM. Page 2 is on disk.

Step 4: Access page not in RAM

- CPU tries to access Page 2 → **Page Fault** occurs.

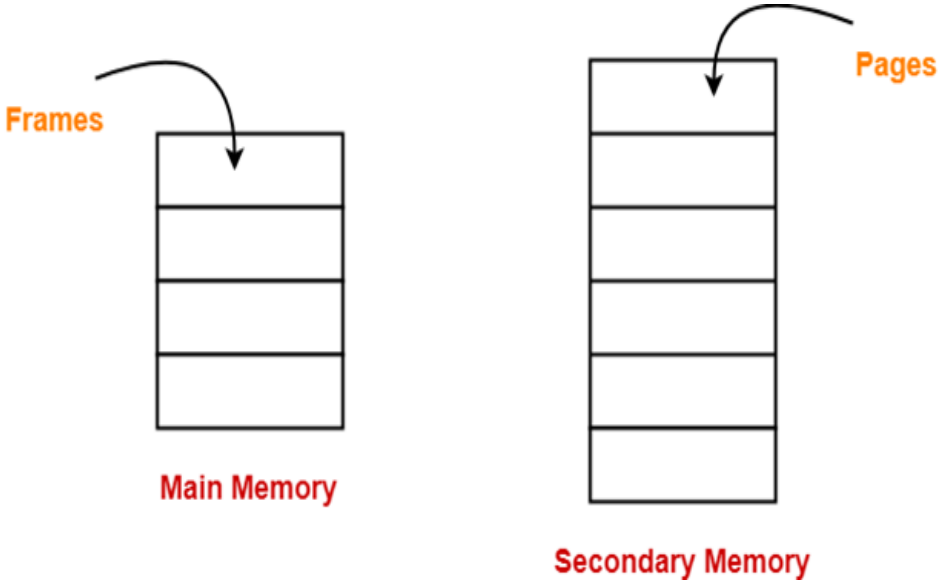
Non-Contiguous Memory Allocation-

- Non-contiguous memory allocation is a memory allocation technique.
- It allows to store parts of a single process in a non-contiguous fashion.
- Thus, different parts of the same process can be stored at different places in the main memory.



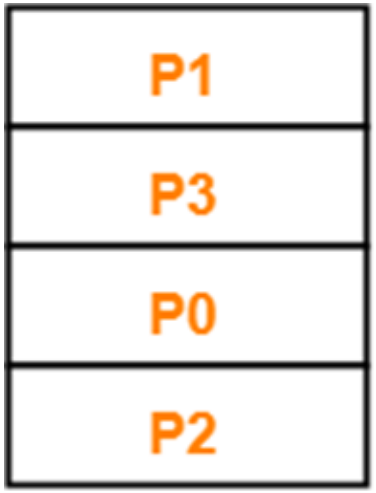
Paging-

- Paging is a fixed size partitioning scheme.
- In paging, secondary memory and main memory are divided into equal fixed size partitions.
- The partitions of secondary memory are called as **pages**.
- The partitions of main memory are called as **frames**.

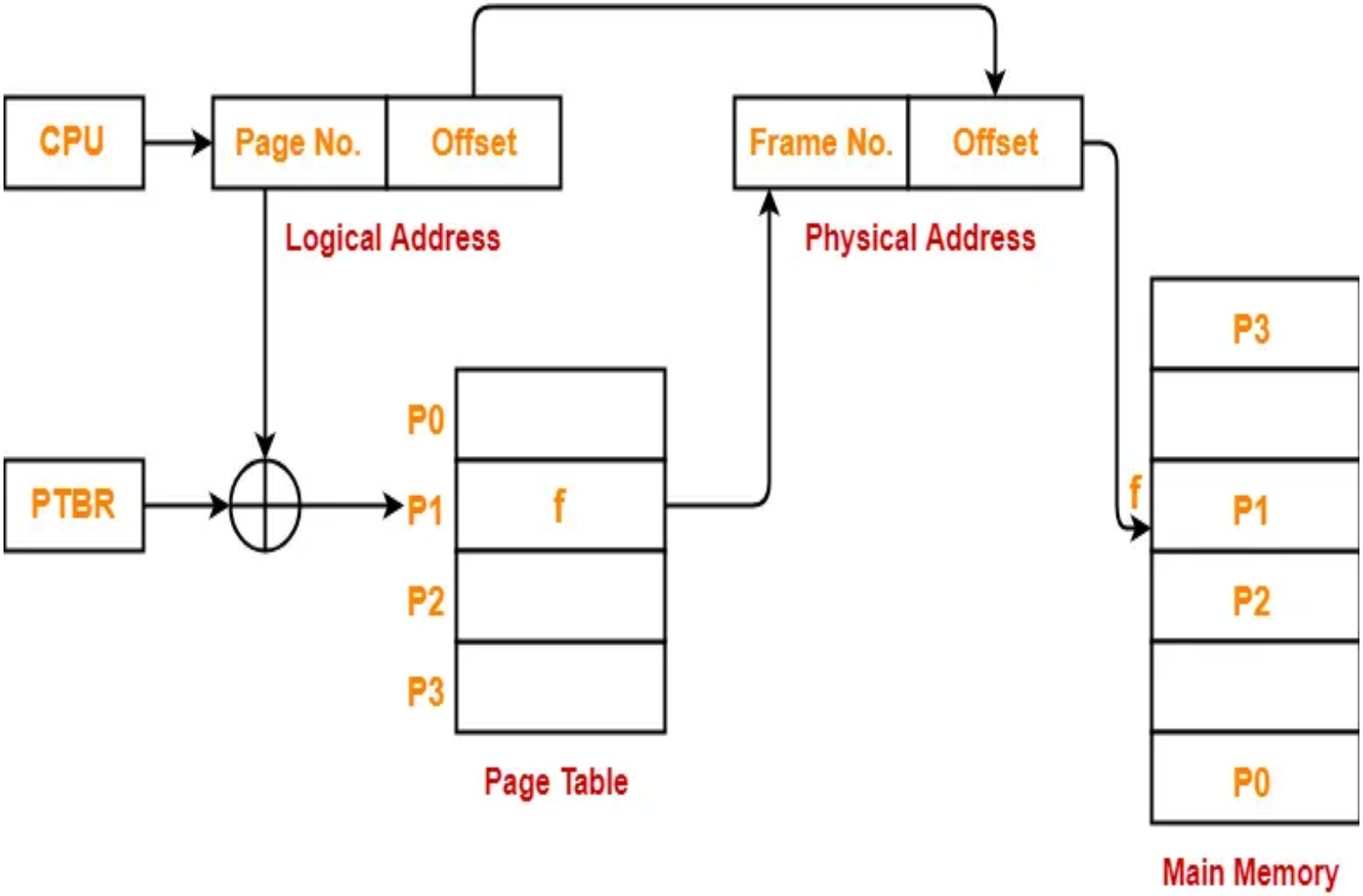


Example-

- Consider a process is divided into 4 pages P_0, P_1, P_2 and P_3.
- Depending upon the availability, these pages may be stored in the main memory frames in a non-contiguous fashion as shown-



Translating Logical Address into Physical Address-



Step 1: CPU Generates Logical Address

- The **CPU** generates a **logical (virtual) address**.
- The logical address has **two parts**:

  Logical Address = Page Number | Offset

- **Page Number** → identifies which page of the process is required
- **Offset** → identifies the exact word/byte within that page

Step 2: Role of PTBR (Page Table Base Register)

- **PTBR** stores the **starting address of the page table** of the currently running process.
- It helps the system locate the correct page table in main memory.

Step 3: Accessing the Page Table

- The **Page Number** from the logical address is added to the **PTBR**.
- This calculation gives the address of the corresponding **Page Table Entry (PTE)**.
- PTBR + Page Number → Page Table Entry

Step 4: Page Table Lookup

- The **Page Table** contains mappings of:
- Page Number → Frame Number
- In the diagram:
  - Pages are labeled **P0, P1, P2, P3**
  - Page **P1** maps to **Frame f**

Step 5: Extracting the Frame Number

- From the page table entry, the **frame number (f)** is obtained.
- This frame number indicates where the required page is located in **main memory**.

Step 6: Formation of Physical Address

- The **physical address** is formed by combining:
- Physical Address = Frame Number | Offset
- **Frame Number** → specifies the frame in main memory
- **Offset** → remains the same as in the logical address
- ✔ Offset is **never changed** during address translation.

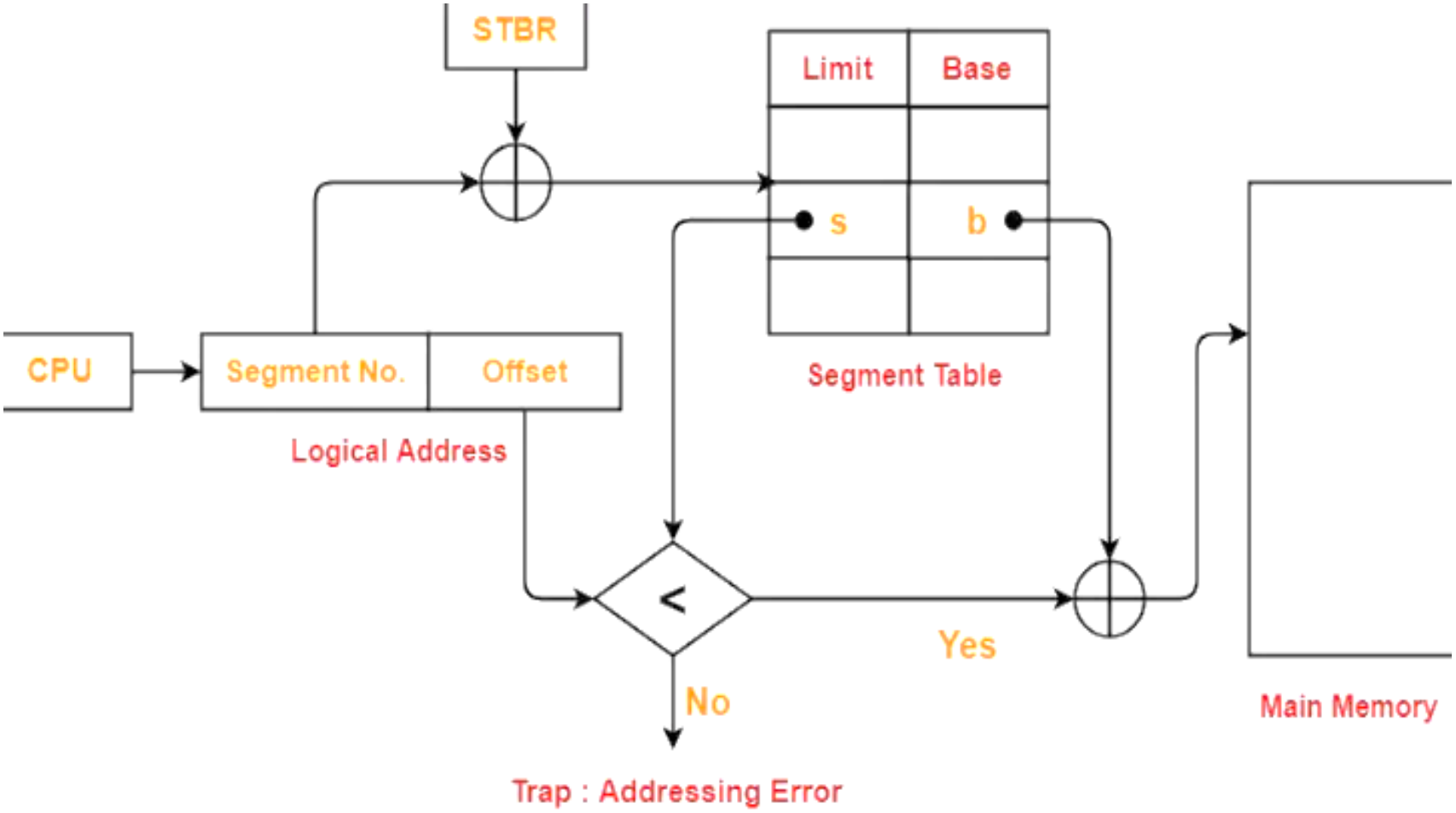
Step 7: Accessing Main Memory

- Using the physical address, the system accesses **main memory**.
- The required data or instruction is fetched from **Frame f** at the given offset.

Segmentation-

- Like Paging, Segmentation is another non-contiguous memory allocation technique.
- In segmentation, process is not divided blindly into fixed size pages.
- Rather, the process is divided into modules for better visualization.
- Segmentation is a variable size partitioning scheme.
- In segmentation, secondary memory and main memory are divided into partitions of unequal size.
- The size of partitions depend on the length of modules.
- The partitions of secondary memory are called as **segments**.

Translating Logical Address into Physical Address-



Step 1:CPU generates a logical address consisting of two parts-

- Segment Number
- Segment Offset

- **Segment Number** specifies the specific segment of the process from which CPU wants to read the data.
- **Segment Offset** specifies the specific word in the segment that CPU wants to read.

Step 2: Segment Table Base Register (STBR)

STBR stores the **starting address of the segment table** in memory.

Segment table contains **Base and Limit** for each segment:

Segment	Base (b)	Limit (l)
s	b	l

Base → starting physical address of the segment
Limit → size of the segment

Step 3: Access Segment Table

- Use **Segment Number** to locate the entry in the segment table:
- Address of segment table entry = STBR + Segment Number

This entry gives:

- **Base address (b)** → start of the segment in physical memory
- **Limit (l)** → maximum valid offset for this segment

Step 4: Check for Valid Offset

- The offset from the logical address is compared with **limit**:
- if Offset < Limit → valid access else → addressing error (trap)
- If **offset exceeds limit**, CPU raises **Trap: Addressing Error**

Step 5: Form Physical Address

- If the offset is valid:
- Physical Address = Base + Offset
- Base → starting location of the segment in main memory
- Offset → exact location within the segment

Step 6: Access Main Memory

- Using the **physical address**, CPU fetches the **required data or instruction** from main memory.